

Managing Application Traffic Using Elastic Load Balancers

<https://github.com/kamalsinghania-void/AWS-Cloud-Lab.git>

<https://github.com/kamalsinghania-void/AWS-Cloud-Lab/tree/13f410bdaec667082eceb883b20dc37d953026d0/ELB>

Getting Started

Elastic Load Balancers serve as a single point of contact and are used to distribute incoming traffic across multiple EC2 instances, IP addresses, or containers, which increases the availability of our applications. AWS supports Network and Application Load Balancers and uses target groups to route the requests it receives.

Network vs. Application Load Balancers

The following table highlights the differences between Network Load Balancers (NLBs) and Application Load Balancers (ALBs):

Network vs. Application Load Balancers

Network Load Balancer	Application Load Balancer
It operates on the network layer (the 4th layer of the OSI model).	It works on the application layer (the 7th layer of the OSI model).
It handles TCP/UDP traffic and provides high throughput and low latency.	It handles HTTP/HTTPS requests and can route requests to Lambda functions.
NLBs use static IP addresses.	ALBs use dynamic IP addresses.
It supports IP addresses and EC2 instances as targets in its target groups.	In addition to IP addresses and EC2 instances, it also supports Lambda functions as targets.
It's ideal for real-time applications, such as video streaming applications.	It's usually used for web applications.

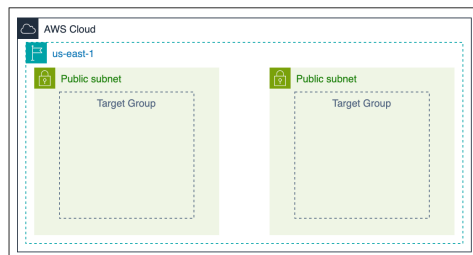
Configure Target Groups

A **load balancer** forwards the requests it receives to target groups, which then route these requests to registered targets, such as EC2 instances using the port and the protocol defined in the target group.

Load balancers use active and passive health checks to monitor our registered targets. In **active health checks**, a load balancer sends connection requests to our targets, whereas in **passive health checks**, the load balancer monitors how the targets respond to incoming connection requests. This makes sure that the targets we've registered are working as expected. If a target fails any of these health checks, its status is changed to **unhealthy**.

In this task, we'll create a target group that will use **TCP** as its protocol and **6000** as its port. We'll later deploy an application server on port **6000** and register it as a target in this target group. We'll also create another target group that will be registered with our ALB.

After the completion of this task, the provisioned infrastructure should look like the one given below:



Architecture diagram

Follow the steps given below to create a target group:

- In the search bar, search for "EC2" and select the "EC2" service.
- Select "Target Groups" under the "Load Balancing" section from the left navigation menu, and click the "Create target group" button.
- In the "Basic configuration" section, select "Instances" and set the name of the target group to **clab-nlb-target-group**.
- Select "TCP" from the drop-down given under "Protocol" and set the "Port" to **6000**.
- Ensure "IPv4" is selected as the IP address type.
- In the "Health checks" section, select "TCP" as the "Health check protocol."
- Expand the "Advanced health check settings" section by clicking it, select "Override," and provide **6000** as the "Health check port." This is the port that will be pinged to check the health of the registered targets.
- Keep the default settings for all other options and click the "Next" button.
- As we don't have any instances that can be registered as targets right now, don't do anything in the "Register targets" page.
- Click the "Create target group" button.

Now, let's follow the steps given above to create another target group that will be used by our ALB with the following changes:

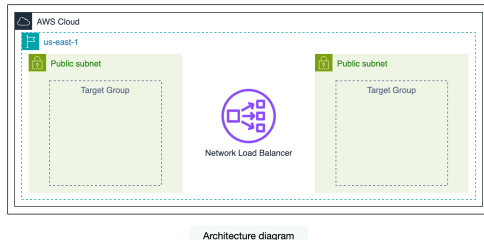
- Set the name of this target group to **clab-alb-target-group**.

- Select “HTTP” as the protocol and **8080** as the port in the “Protocol : Port” section.
- Ensure “HTTP” is selected as the “Health check protocol.”
- In the “Advanced health check settings” section, select “Override” and provide **8080** as the “Health check port.”
- Click the “Next” button and click the “Create target group” button on the “Register targets” page.

Create a Network Load Balancer

Now that we’ve created a target group, let’s create an NLB in this task. The NLB uses listeners that check for connection requests using the port and protocol we define. When a client accesses the load balancer, the NLB analyzes their request and redirects it to a target registered within the target group.

After the completion of this task, the provisioned infrastructure should look like the one given below:



Follow the steps given below to create an NLB:

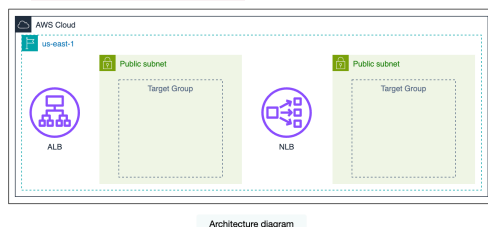
- On the EC2 dashboard, select “Load Balancers” under the “Load Balancing” section from the left navigation menu and click the “Create load balancer” button.
- Click the “Create” button given in the “Network Load Balancer” section.
- Set the name of the load balancer to **clab-nlb**.
- Select “Internal” as the “Scheme” and ensure “IPv4” is selected as the “Load balancer IP address type.”
- In the “Network mapping” section, do the following:
 - Keep the default VPC selected as we need to create the load balancer in the same VPC where the instances will be launched.
 - Select the **us-east-1a** as the Availability Zone under the “Mappings” section. Each mapping corresponds to a separate subnet in a different Availability Zone. The load balancer will route traffic to targets within the selected Availability Zone and subnet.
- In the “Security Groups” section, remove any security groups that are preselected.
- In the “Listeners and routing” section, do the following:
 - Ensure “TCP” is selected as the protocol.
 - Set the port to **6000**.
 - Select the target group, “clab-nlb-target-group” from the drop-down available under the “Default action” option.
 - Keep the default settings for all other options and click the “Create load balancer” button.

Move back to the “Load Balancers” page, where our newly created load balancer should be visible in the list.

- On the “Load Balancers” page, select our load balancer and copy and keep the following somewhere safe because we’ll be using them later:
 - The “DNS name” under the “Details” tab.
 - The “IPv4 VPC CIDR” under the “Network mapping” tab.

Create an Application Load Balancer

An **Application Load Balancer** is used to distribute incoming web traffic across multiple IP addresses or EC2 instances. In this task, we’ll create an ALB that’ll use the **clab-alb-target-group** to distribute the traffic it receives. After the completion of this task, the provisioned infrastructure should look like the one given below:



Create a security group

Before we create an ALB, let’s create a security group that will be associated with our load balancer, by following the steps given below. This security group will control the traffic allowed to and from our load balancer and will secure access to our application by defining inbound and outbound rules.

- On the EC2 dashboard, select “Security Groups” under the “Network & Security” section from the left navigation bar.
- Click the “Create security group” button.
- In the “Basic details” section, do the following:
 - Set **clab-alb-sgr** as the name of the security group.
 - In the description of the group, write **security group for ALB**.
 - In the “VPC” section, ensure the default VPC is selected

- In the "Inbound rules" section, ensure the default rule is selected.
- Now, let's add the following inbound rule to our security group by clicking the "Add rule" button under the "Inbound rules" section:
 - Type: **Custom TCP**
 - Protocol: **TCP**
 - Port range: **8080**
 - Source: **Anywhere-IPv4**
- Ensure a rule that allows all traffic is added in the "Outbound rules" section, and then click the "Create security group" button.

Create an ALB

Once our security group is created, follow the steps given below to create an ALB:

- On the EC2 dashboard, select "Load Balancers" from the left navigation menu and click the "Create load balancer" button.
- Click the "Create" button in the "Application Load Balancer" section.
- Set **clab-alb** as the name of the load balancer.
- Select "Internet-facing" as the "Scheme" and ensure "IPv4" is selected as the "Load balancer IP address type".
- In the "Network mapping" section, do the following:
 - Select the Availability Zones **us-east-1a** and **us-east-1b** under the "Mappings" section
- In the "Security Groups" section, select the security group **clab-alb-sgr** from the available drop-down list, and unselect the default security group if it's preselected.
- In the "Listeners and routing" section, do the following:
 - Ensure "HTTP" is selected as the protocol.
 - Set the port to **8080**.
 - Select the **clab-alb-target-group** target group from the drop-down available under the "Default action" option.
- Keep the default settings for all other options and click the "Create load balancer" button.

After the load balancer is created, make sure to copy its DNS address and keep it somewhere safe, as we'll be using it later.

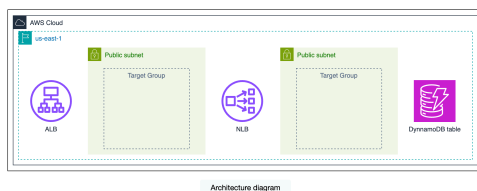
Congratulations! You've launched an ALB and an NLB. Now, let's see these load balancers in action in the upcoming tasks.

Create a DynamoDB Table



DynamoDB is a NoSQL database service provided by Amazon Web Services (AWS). It's a key-value and document-based database that's fully managed and provides high performance at scale. It provides us with high scalability, reliability, built-in security, data restoration, and data availability without any dataset limit.

DynamoDB stores data in the form of tables. These tables contain datasets as items. In this task, we'll create a table in DynamoDB that will store information about users, such as their names and email addresses. This information will later be fetched by our back-end application. After completing this task, the provisioned infrastructure should look like the one given below:



Create a table

Follow the steps given below to create a DynamoDB table:

- In the search bar, search for "DynamoDB" and select "DynamoDB" from the search results. This redirects us to the DynamoDB dashboard.
- On the DynamoDB dashboard, click the "Create table" button. This will open the "Create table" page.
- On the "Create table" page, enter the following values:
 - Enter the table name as **clab-dynamodb-table**.
 - Add a partition key **UserID** and set its data type as "String."
- Let the other settings remain as they are and click the "Create table" button.

We'll be redirected to the "Tables" page. Our new "clab-dynamodb-table" table should be visible in the list with the status "Creating." This status should change to "Active" in a few seconds.

Add items to the table

Now that we've created a table, let's add an item to it by following the steps given below:

- Navigate to the "DynamoDB > Tables > clab-dynamodb-table" page.
- In the drop-down list available under the "Actions" section, click the "Create item" button.
 - Set `client-01` as the "Value" of "UserID."
 - In the drop-down list available under the "Add new attribute", click the "String" option.
 - Set the "Attribute name" to `Name` and set its "Value" to any name of your choice.
 - Once again, in the drop-down list available under the "Add new attribute" section, click the "String" option.
 - Set the "Attribute name" to `Email`, and provide an email address as its "Value."
- Click the "Create item" button.

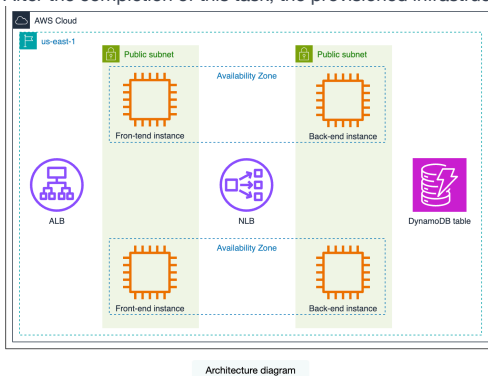
Once this item is created, follow the steps given above to create another item with the following details:

- The "Value" of "UserID" as `client-02`.
- Add an attribute with `Name` as the "Attribute name" as a name of your choice as its "Value."
- Add an attribute with `Email` as the "Attribute name" as an email of your choice as its "Value."

Launch the EC2 Instances

Amazon Elastic Compute Cloud (EC2) provides us with computing capacity inside the AWS Cloud. In this task, we'll launch four instances: two will be used to deploy our front-end application and two will be used to deploy our back-end application. We'll then register these instances as targets in `clab-alb-target-group` and `clab-nlb-target-group`.

After the completion of this task, the provisioned infrastructure should look like the one given below:



Create a security group for back-end instances

Before we create our EC2 instances, let's create security groups with the following inbound rules that will be associated with them:

- On the EC2 dashboard, select "Security Groups" from the left navigation bar and click the "Create security group" button.
- In the "Basic details" section, do the following:
 - Set `clab-backend-sgr` as the name of the security group.
 - In the description of the group, write `security group for backend`.
 - In the "VPC" section, ensure the default VPC is selected.
- Now, let's add the following three inbound rules to our security group by clicking the "Add rule" button:
 - Type: `SSH`, Protocol: `TCP`, Port range: `22`, Source: `Anywhere-IPv4`
 - Type: `Custom TCP`, Protocol: `TCP`, Port range: `6000`, Source: `Anywhere-IPv4`
 - Type: `All traffic`, Protocol: `All`, Port range: `All`, Source: `Custom`, and provide the IP address range of the NLB we created previously
- Ensure a rule that allows all traffic is added in the "Outbound rules" section and then click the "Create security group" button.

Create a security group for front-end instances

Now, create another security group by repeating the steps given above, set its name to `clab-frontend-sgr`, and provide the following inbound rules:

- Type: `SSH`, Protocol: `TCP`, Port range: `22`, Source: `Anywhere-IPv4`
- Type: `Custom TCP`, Protocol: `TCP`, Port range: `8080`, Source: `Anywhere-IPv4`
- Type: `All traffic`, Protocol: `All`, Port range: `All`, Source: `Custom`, and select "clab-alb-sgr" given under the "Security Groups" section from the available drop-down list
- Ensure a rule that allows all traffic is added in the "Outbound rules" section and then click the "Create security group" button.

Create the back-end instances

Once our security groups are created, let's follow the steps given below to create our EC2 instances:

- On the EC2 dashboard, select "Instances" from the left navigation bar and click the "Launch instances" button.
- Set the name of the instance to `clab-backend-instance` in the "Name and tags" section.
- In the "Application and OS Images (Amazon Machine Image)" section, select "Amazon Linux 2023 AMI" as the "Amazon Machine Image (AMI)" and ensure "64-bit (x86)" is selected as the architecture.
- Select "t2.micro" in the "Instance type" section.
- Select "Proceed without a key pair (Not recommended)" in the "Key pair (login)" section.
- In the "Network settings" section, click "Edit" and select the subnet whose Availability Zone is `us-east-1a` from the drop-down available under the "Subnet" section.
- In the "Firewall (security groups)" section, click "Select existing security group" and select "clab-backend-sgr" from the drop-down menu.
- Ensure `8 GiB` is selected as the storage and `gp3` is selected as the root volume in the "Configure storage" section.
- In the "Advanced details" section, provide the following script in the "User data" subsection:

```
#!/bin/bash
```

```
#install the required packages
```

```
sudo su
```

```
yum update -y
```

```
yum install golang-go -y
```

- In the "Summary" section, set `2` as the number of instances.
- Review the settings for our instance and click the "Launch instance" button.
- Click the "View all instances" button to move to the "Instances" page.

Once both of the back-end instances are launched, change the name of the back-end instances to `clab-backend-instance-01` and `clab-backend-instance-02` by clicking the pencil icon in the "Name" section on the "EC2 > Instances" page.

Create the front-end instances

Now, repeat the steps given above to create two more instances, which we'll use to deploy our front-end application with the following settings:

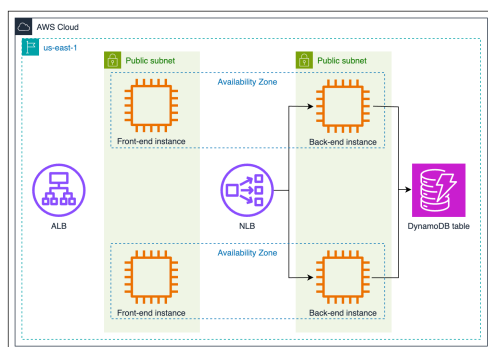
- Set the name of this instance to `clab-frontend-instance`.
- In the "Network settings" section, select "clab-frontend-sgr" from the drop-down menu.

Once both of our front-end instances are launched, change the name of the front-end instances to `clab-frontend-instance-01` and `clab-frontend-instance-02` by clicking the pencil icon in the "Name" section on the "EC2 > Instances" page.

Set Up the Back-End Server

Now that we've created our load balancers, target groups, and EC2 instances, let's register our back-end instances as targets in our `clab-nlb-target-group`. Once this instance becomes a registered target, our NLB will direct any connection requests it receives to our server.

After the completion of this task, the provisioned infrastructure should look like the one given below:



Architecture diagram

Deploy back-end application

Let's first deploy our server application on our back-end instances by following the steps given below:

- Navigate to "EC2 > Instances," select one of the back-end instances, and click the "Connect" button.
- On the pop-up page, click the "Connect" button. This will open an EC2 terminal in a new browser tab.
- Copy and paste the following commands in the EC2 terminal to install `aws-sdk` for Go and create a back-end application:

```
go mod init aws
```

```
cd go
go get -u github.com/aws/aws-sdk-go
nano backend.go
```

A text editor will open up once the commands given above are executed. Here, copy and paste the back-end application given below and save the content by pressing “Ctrl X” (or “Control X”), then the “Y” key, and finally press the “Enter” key to exit the nano editor.

```
package main

import (
    "fmt"
    "log"
    "net"
    "github.com/aws/aws-sdk-go/aws"
    "github.com/aws/aws-sdk-go/aws/credentials"
    "github.com/aws/aws-sdk-go/aws/session"
    "github.com/aws/aws-sdk-go/service/dynamodb"
)

func main() {
    ln, err := net.Listen("tcp", ":6000")
    if err != nil {
        log.Fatal(err)
    }

    for {
        conn, err := ln.Accept()
        if err != nil {
            log.Println(err)
            continue
        }
        go handleConnection(conn)
    }
}

func handleConnection(c net.Conn) {
    log.Println("A client has connected", c.RemoteAddr())
    defer c.Close()

    // Buffer to store incoming data
    buffer := make([]byte, 1024)

    // Read data from the client
    n, err := c.Read(buffer)
    if err != nil {
        fmt.Println("Error reading data from client:", err)
        return
    }

    // Print the received data
    client_value := string(buffer[:n])
    fmt.Printf("Received data from client: %s\n", client_value)
    sess := session.Must(session.NewSession(&aws.Config{
        Region:    aws.String("us-east-1"), // Replace with your AWS region
        Credentials: credentials.NewStaticCredentials("AKIAWHRR72TTEAHQ4LY2", "LYUHavuXaOjZntdFDhXCc9ZkwsdZo5yoZIKPEI6R", ""),
    }))

    // Create a DynamoDB client.
    dynamodb_client := dynamodb.New(sess)

    table_name := "clab-dynamodb-table"
    key_name := "UserID"
    key_value := client_value

    input := &dynamodb.GetItemInput{
        TableName: aws.String(table_name),
        Key: map[string]*dynamodb.AttributeValue{
            key_name: {
                S: aws.String(key_value),
            },
        },
    }

    result, err := dynamodb_client.GetItem(input)
    if err != nil {
        fmt.Println("Error getting item from DynamoDB", err)
        return
    }

    // Check if the item was found.
    if result.Item == nil {
        fmt.Println("Item not found")
        return
    }

    // Process the retrieved item.
    var tabledata []string
    for key, value := range result.Item {
        // fmt.Printf("%s: %v\n", key, value)
        formattedString := fmt.Sprintf("%s: %v\n", key, value)
        tabledata = append(tabledata, formattedString)
    }
    messageforclient := ""
    for _, str := range tabledata {
        messageforclient += str
    }

    c.Write([]byte(messageforclient))
}
```

Now, let's run the command given below to deploy our back-end application:

```
go run backend.go
```

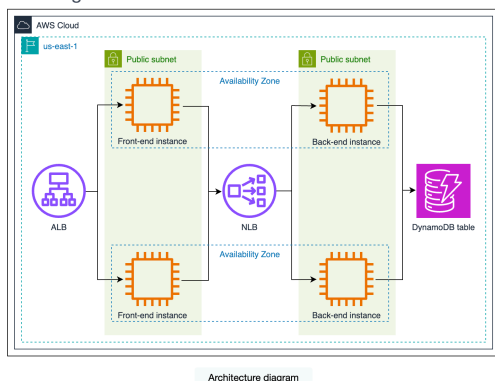
Register back-end instances in the target group

Now that our back-end application is deployed on both of our back-end instances, let's register these instances as a target in our target group by following the steps given below:

- Navigate to "EC2 > Target Groups" and select `clab-nlb-target-group`.
- Click the "Register targets" button in the "Targets" tab.
- On the "Register targets" page, select the instance "clab-backend-instance-01" and "clab-backend-instance-02" from the "Available instances" section and click the "Include as pending below" button.
- Click the "Register pending targets" button to register these instances as a target.

Set Up the Front-End Application

Now that our back-end application is up and running, let's launch a front-end application that will send a connection request to this backend using our NLB. We'll then register these front-end instances as targets in our `clab-alb-target-group`. After the completion of this task, the provisioned infrastructure should look like the one given below:



Deploy front-end application

Follow the steps given below to deploy our front-end application on the `clab-frontend-instance-01`:

- Navigate to "EC2 > Instances," "select the "clab-frontend-instance-01" instance, and click the "Connect" button.
- On the pop-up page, click the "Connect" button. This will open an EC2 terminal in a new browser tab.
- Copy and paste the following command in the EC2 terminal:

```
nano frontend.go
```

Once the command given above is executed, a nano text editor will open up.

- Here, provide the DNS name of our NLB on **line 18** and then copy and paste the following code in the editor.
- Save the file by pressing "Ctrl X" (or "Control X"), then the "Y" key, and finally press the "Enter" key to exit the nano editor.

```
package main
import (
    "fmt"
    "html/template"
    "log"
    "net"
    "net/http"
)
type PageData struct {
    Message string
}
func main() {
    handler := func(w http.ResponseWriter, r *http.Request) {
        // Send a connection request to the load balancer
        conn, err := net.Dial("tcp", "<Provide NLB DNS name here>:6000")
        if err != nil {
            log.Println("We failed to connect to the back-end server")
            http.Error(w, "Internal Server Error", http.StatusInternalServerError)
            return
        }
        defer conn.Close()
        // Set the value we'll send to the backend
        dynamodbValue := "client-01"
        _, err = conn.Write([]byte(dynamodbValue))
        if err != nil {
            log.Println("Error writing to connection:", err)
            http.Error(w, "Internal Server Error", http.StatusInternalServerError)
            return
        }
        // Receive data from backend
        recvdSlice := make([]byte, 150)
        n, err := conn.Read(recvdSlice)
        if err != nil {
            log.Println("Error reading from connection:", err)
            http.Error(w, "Internal Server Error", http.StatusInternalServerError)
            return
        }
    }
}
```

```

    }
    data := PageData{
        Message: string(recvdSlice[:n]),
    }
}
// Create an HTML template
tmpl, err := template.New("index").Parse(`
<DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Hello, Go!</title>
<style>
body {
font-family: fantasy;
background-color: #17202D;
margin: 0;
display: flex;
justify-content: center;
align-items: center;
height: 100vh;
}
.container {
text-align: center;
}
}
.message {
font-weight: bold;
color: lightyellow;
}
</style>
</head>
<body>
<div class="container">
<h1 class="message">Congratulations!</h1>
<h2 class="message">You've connected to the back-end server and fetched the following data</h2>
<h3 class="message">{{.Message}}</h3>
</div>
</body>
</html>
`)
    if err != nil {
        log.Println("Error parsing template:", err)
        http.Error(w, "Internal Server Error", http.StatusInternalServerError)
        return
    }
    // Execute the template with the provided data
    err = tmpl.Execute(w, data)
    if err != nil {
        log.Println("Error executing template:", err)
        http.Error(w, "Internal Server Error", http.StatusInternalServerError)
    }
}
// Register the handler function for the root URL path "/"
http.HandleFunc("/", handler)
// Start the HTTP server on port 8080
fmt.Println("Server is listening on :8080")
err := http.ListenAndServe(":8080", nil)
if err != nil {
    fmt.Println("Error starting server:", err)
}
}

```

Now, let's deploy our frontend.go file on port 8080 by executing the command given below:

```
go run frontend.go
```

- Once the command given above is executed, our front-end application will send a connection request to our back-end application server. If the connection is successful, we'll receive a message from our host.

Follow the steps given above to deploy a front-end application on the `c1ab-frontend-02`. While creating the `frontend.go` file, change the value of `dynamodbValue` to `client-02` on **line 27**.

Register front-end instances in the target group

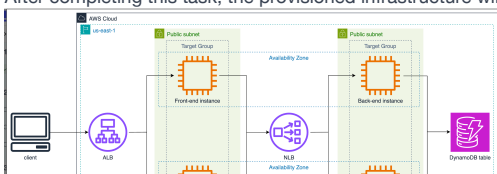
Now that our front-end application has been deployed on both of our instances, let's follow the steps given below to add them as targets in our ALB target group:

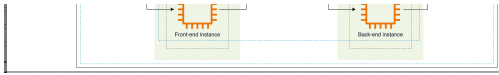
- Navigate to "EC2 > Target Groups" and select `c1ab-alb-target-group`.
- Under the "Targets" tab, click the "Register targets" button.
- On the "Register targets" page, select both of the front-end instances from the "Available instances" section and click the "Include as pending below" button.
- Click the "Register pending targets" button to register these targets.

View the Load Balancers in Action

Now that we've created our load balancers, deployed our front- and back-end applications, and registered our targets, let's test our network and application load balancers. Although we won't be creating any new resources in this task, we'll make a request to our load balancers and carry out some tests.

After completing this task, the provisioned infrastructure will be similar to the one shown in the figure below:





Test the ALB

- On the “EC2 > Target Groups > clab-alb-target-group” page, open the “Targets” tab and wait for the health of any one of the targets to change to **Healthy**.
- Please copy and paste the following URL in a new browser tab, and don’t forget to provide the DNS address of the ALB we created earlier. This will display the data our front-end application received from our back-end server.

<http://<Provide ALB DNS Name Here>:8080/>

We should see the information of the user, depending on the instance our request was directed to. If the ALB sent our request to “clab-frontend-instance-01,” we’ll see the information for **client-01**, and if it sent our request to “clab-frontend-instance-02,” we’ll see the information for **client-02**.

Try making some more requests by refreshing the page, and you should see the username changing as the load balancer directs traffic to different instances.

Test the ALB in case of instance failure

Now, let’s see how our load balancers will behave in case one of our instances fails. To do this, perform the following:

- Close the browser tab where one of the front-end application has been deployed.
- Now, close the browser tab where one of the back-end application has been deployed.

By doing this, our instances will be in a “Running” state; however, our front- and back-end applications will shut down. When this happens, these instances will fail the health checks done by our load balancers.

Let’s view the status of the registered targets once again. Return to the details page of our **clab-nlb-target-group** target group. After a few minutes have passed, the health status of one of the instances should change to **unhealthy** once it fails two consecutive health checks. The status should then change to **draining** as it gets deregistered and taken out of service.

Notice the same behavior in the **clab-alb-target-group** target group.

Let’s check if our application is still available on the DNS address of our ALB by opening the URL given at the beginning of this task in a new browser window. We’ll be able to see our front-end application in this browser tab if everything is working as expected.

Clean Up

The final task is to remove all the resources we created. Follow the steps given under each task to delete the resources we created.

Delete the Elastic Load Balancers

- Navigate to “EC2 > Load Balancers” and select the load balancers we created earlier.
- In the “Actions” drop-down menu, click the “Delete load balancer” button.
- In the confirmation pop-up, type **confirm** and click the “Delete” button.

Delete target groups

- Navigate to “EC2 > Target Groups” and select the target groups we created earlier.
- In the “Actions” drop-down menu, click the “Delete” button.
- In the confirmation pop-up, click the “Yes, delete” button.

Terminate the EC2 instances

- Navigate to “EC2 > Instances.”
- Select all the instances we created earlier and click the “Instance state” option.
- From the drop-down, select the “Terminate instance” option.
- Click the “Terminate” button in the prompt box.

Delete security groups

- Navigate to “EC2 > Security Groups” and select all the security groups we created earlier.
- Click “Actions” and select the “Delete security groups” option from the drop-down menu.
- In the confirmation pop-up, enter **delete** and click the “Delete” button.

Note: You won’t be able to delete the security groups associated with the ALB and EC2 instances till these resources are deleted.

Delete the DynamoDB table

- Navigate to “DynamoDB > Tables” and select the table we created earlier.
- Click the “Delete” button.
- In the confirmation pop-up, type **confirm** and then click the “Delete” button.